

Explaining Main.java

StudentManagementSystem — com.registry.Main

Source code

```
package com.registry;

import com.registry.io.FileManager;
import com.registry.service.StudentService;
import com.registry.ui.ConsoleMenu;

public class Main {
    public static void main(String[] args) {
        FileManager fileManager = new FileManager("students.csv");
        StudentService service = new StudentService(fileManager);
        ConsoleMenu menu = new ConsoleMenu(service);
        menu.run();
    }
}
```

What it does

This is the entry point of the application — the method the JVM calls first when the program starts. It wires together the three main layers of the app and then hands control to the UI loop:

- **1. Create the FileManager**
FileManager fileManager = new FileManager("students.csv");
Points the persistence layer at the file **students.csv**. This object is responsible for reading and writing student records to disk.
- **2. Create the StudentService**
StudentService service = new StudentService(fileManager);
Wraps the FileManager in a service object that holds the business logic — adding, finding, and removing students — delegating actual storage to the FileManager.
- **3. Create the ConsoleMenu**
ConsoleMenu menu = new ConsoleMenu(service);
Builds the interactive text-based user interface around the service, so menu choices translate into calls on StudentService.
- **4. Run the menu**
menu.run();
Starts the menu loop. This is where the program spends essentially all its runtime: prompting for input and dispatching to the service until the user exits.

A note on decompiled variable names (var3)

When you open the compiled **Main.class** file in VS Code with a decompiler extension, the local variables show up as **var0**, **var1**, **var2**, **var3** instead of their original names. This happens because the Java compiler strips local variable names from the bytecode by default (unless compiled with **javac -g**

debug symbols) — only the variable's type and storage slot number are needed for the JVM to run the program.

The decompiler has no way to recover the original names, so it falls back to generic names in slot order. In this method, **var1** corresponds to **fileManager**, **var2** to **service**, and **var3** to **menu**. So **var3.run()** in the decompiled view is really just **menu.run()** from the original source.

Takeaway: for reading and editing logic, always work from the .java files under **src/** — the decompiled .class view is only useful when you don't have the source and need to reverse-engineer what compiled code does.